

Nakijk 02_01 — V2

NAKIJK V2 · Hoofdstuk 02 · Model, lijst en statuscodes · bij Oefening_02_01.md · tweede versie van je code, 9 oktober 2026

Nagekeken: `ProgrammingAdvancedOefeningen` → `Controllers/ArtikelController.cs` (gewijzigd op 9 oktober om 20:58) en `Models/Artikel.cs` (niet gewijzigd sinds V1).

OPMERKING

Dit is een nakijk van jouw code, geen oplossing. Er staat geen verbeterde code in.

Ik kon je API ook nu niet starten: op de werkruimte bij je laptop staat geen .NET. Ik heb je code gelezen en elk verzoek uit de opgave op papier nagelopen. De regels van ASP.NET waarop ik me baseer, heb ik nagekeken in de documentatie van Microsoft.

Deze versie legt meer uit dan V1, zoals je vroeg. Bij elke fout staat waarom het misloopt, welk begrip erachter zit, wat er goed is aan je aanpak en hoe je het zelf kan controleren. Achteraan staan de vragen die je waarschijnlijk stelt, met hun antwoord.

In één oogopslag

Je hebt twee dingen aangepast: de routes van punt 4 en 5, en de controle van punt 5.

- De controle van punt 5 is een stap in de goede richting: je kijkt nu naar het aantal in de lijst, niet meer naar de lijst zelf. Je vergelijkt dat aantal alleen nog met het verkeerde.
- De nieuwe routes lossen punt 4 niet op, en ze breken punt 5, dat in V1 voor een bestaande soort wel werkte. Beide routes beginnen nu met `/`, en dat heeft een speciale betekenis. Het eerste deel hieronder legt uit hoe ASP.NET een adres samenstelt. Lees dat eerst: A1 en A5 volgen eruit.

Punt	Wat de opgave vraagt	V1	V2
1	het model <code>Artikel</code>	✓ in orde	✓ in orde
2	de controller op <code>api/artikel</code> met minstens zes artikelen	✓ in orde	✓ in orde
3	GET <code>api/artikel</code>	✓ in orde	✓ in orde
4	GET <code>api/artikel/{id}</code>	✗ verkeerde route	✗ nog steeds een verkeerde route (A1)
5	GET <code>api/artikel/soort/{soort}</code>	½ onbekende soort geeft 200	✗ werkt niet meer (A5), en de onbekende soort nog niet (A2)
6	GET <code>api/artikel/voorradijg</code>	✗ ontbreekt	✗ ontbreekt (A3)
7	<code>ArtikelFiche</code> en GET <code>api/artikel/fiche/{id}</code>	✗ ontbreekt	✗ ontbreekt (A4)

Fout uit V1	Status in V2
A1, punt 4: verkeerde route	nog open; de route is veranderd, maar komt nog altijd niet uit op <code>api/artikel/{id}</code>
A2, punt 5: onbekende soort geeft 200	nog open, maar dichterbij de oplossing
A3, punt 6 ontbreekt	nog open
A4, punt 7 ontbreekt	nog open
B1, returntype van <code>GetArtikelSoort</code>	nog open
A5, punt 5: soortroute	nieuw in V2

Eerst: hoe ASP.NET een adres samenstelt

Dit is het begrip achter A1 en A5. Het staat in les 02, bij de routes, en in de documentatie van Microsoft onder *Combining attribute routes*.

Het adres van een endpoint bestaat uit twee stukken: de route op de **controller** en de route op de **methode**. ASP.NET plakt ze aan elkaar volgens vier regels:

1. **De controllerroute.** Op je controller staat `api/[controller]`. Het stuk `[controller]` wordt vervangen door de naam van je klasse zonder het woord *Controller*. Bij jou wordt dat `api/Artikel`.
2. **Een methode zonder eigen route** krijgt gewoon het adres van de controller. Zo werkt jouw `GetArtikelen`: dat luistert op `api/Artikel`, en daarom is punt 3 in orde.
3. **Een methode met een route die niet met ``/`` begint**, krijgt die route áchter het adres van de controller, met een `/` ertussen. Staat er op een controller `winkel` en bij een methode `kassa`, dan is het adres `winkel/kassa`.
4. **Een methode met een route die wél met ``/`` (of met ``~/``) begint**, is een absoluut adres. Dan wordt de route van de controller **niet** gebruikt. Met `kassa` voorafgegaan door een `/` is het adres gewoon `kassa`, zonder `winkel` ervoor. Microsoft zegt het letterlijk: *Route templates applied to an action that begin with / or ~/ don't get combined with route templates applied to the controller*.

Twee dingen die daarbij goed zijn om te weten:

- **Hoofdletters spelen geen rol** in een adres. `api/Artikel` en `api/artikel` zijn hetzelfde.
- **Een verkeerde route geeft geen foutmelding** bij het bouwen. Elk adres is voor C# geldig. Je merkt het pas als je het adres uit de opgave oproept en er niets antwoordt.

Toegepast op jouw code:

Methode	Route op de methode	V1: volledig adres	V2: volledig adres	De opgave vraagt
GetArtikelen	geen	<code>api/Artikel</code>	<code>api/Artikel</code>	<code>api/artikel</code> ✓
GetArtikelId	V1: <code>"artikel/{id}"</code> · V2: <code>"/artikel/artikel/{id}"</code>	<code>api/Artikel/artikel/{id}</code>	<code>artikel/artikel/{id}</code>	<code>api/artikel/{id}</code>
GetArtikelSoort	V1: <code>"soort/{soort}"</code> · V2: <code>"/artikel/soort/{soort}"</code>	<code>api/Artikel/soort/{soort}</code> ✓	<code>artikel/soort/{soort}</code>	<code>api/artikel/soort/{soort}</code>

TIP

Je hoeft dit niet in je hoofd uit te rekenen. Start je API en open Scalar: daar staat bij elk endpoint het volledige adres. Leg dat naast het adres in de opgave. Ze moeten letter voor letter gelijk zijn, behalve de hoofdletters.

A. Wat een verkeerd antwoord geeft

A1. Punt 4: het endpoint luistert nog altijd op een ander adres

Waar: `ArtikelController.cs`, het attribuut boven `GetArtikelId` (regel 25).

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>api/artikel/1</code>	404 Not Found, zonder bericht	200 OK met artikel 1
GET <code>api/artikel/99</code>	404 Not Found, zonder bericht	404 Not Found met <i>We vonden geen artikel met Id 99.</i>

Wat er veranderd is. In V1 kwam je route áchter de controllerroute (regel 3 hierboven), en luisterde je endpoint op `api/Artikel/artikel/{id}`. Nu begint je route met een `/` (regel 4). Daardoor valt de controllerroute weg, en luistert je endpoint op `artikel/artikel/{id}`, zonder `api` ervoor. Het is dus een ander verkeerd adres geworden.

Waarom je twee keer dezelfde lege 404 krijgt. Een verzoek naar `api/artikel/1` past bij geen enkel endpoint. Dan bereikt het je code niet eens: ASP.NET zelf antwoordt met een lege 404. Je eigen bericht op regel 33 staat goed, maar wordt nooit uitgevoerd. Dat is ook waarom een bestaand en een onbestaand Id hetzelfde antwoord geven.

Richting.

- Schrijf voor jezelf op welk stuk van `api/artikel/{id}` de controller al levert. Het stuk dat overblijft, is wat de methode er nog bij moet zetten, volgens regel 3.
- Vergelijk met je theorieproject: de `ProductController` van de les heeft een endpoint dat één laptop op zijn Id ophaalt. Kijk hoe de route daar geschreven is.
- Controleer in Scalar: het adres moet `api/Artikel/{id}` zijn.

A2. Punt 5: een soort die niet bestaat, geeft nog altijd geen 404

Waar: `ArtikelController.cs`, de voorwaarde in `GetArtikelSoort` (regel 53).

Verzoek (na het oplossen van A5)	Je API antwoordt dan	De opgave vraagt
GET <code>api/artikel/soort/processor</code>	200 OK met <code>[]</code>	404 Not Found met <i>We vonden geen artikelen van de soort processor.</i>

Wat goed is aan je aanpak. Een lege lijst aanmaken, er in de lus de passende artikelen in stoppen, en na de lus controleren: dat is een goede manier om dit op te lossen. En je hebt gelijk dat de lijst vóór de lus moet staan. Dat heet **scope**: een variabele bestaat enkel binnen het blok tussen accolades waarin je ze aanmaakt.

- Maak je de lijst **binnen de lus**, dan bestaat ze enkel tijdens één ronde van de lus. Na de lus is ze weg, en dan kan je ze niet controleren. Bovendien zou je in elke ronde een nieuwe, lege lijst krijgen.
- Maak je ze **in de methode, vóór de lus**, zoals je deed, dan bestaat ze zolang de methode loopt. Dat is precies wat je nodig hebt.
- Wat jij *globaal* noemt, is in C# eerder het **veld** bovenaan je klasse, `Artikelen` op regel 8. Dat bestaat zolang de controller bestaat, en elke methode kan erbij. Voor deze lijst is dat niet nodig, en ook niet wenselijk: ze hoort enkel bij dit ene verzoek.

Wat er veranderd is. In V1 vroeg je of de lijst `null` is. Nu vraag je of het **aantal** `null` is. Dat is een stap vooruit: het aantal is wel degelijk wat je moet controleren.

Waarom het nog niet werkt. `Count` geeft een `int`: een geheel getal. Een `int` is een waardetype en kan nooit `null` zijn. Bij een lege lijst is `Count` gewoon het getal dat "niets" betekent. Je vergelijking is dus altijd onwaar, en de 404 wordt nooit bereikt.

De compiler zag dat ook. Je build geeft op regel 53 deze waarschuwing:

CS0472: The result of the expression is always 'false' since a value of type 'int' is never equal to 'null' of type 'int?'.

Vrij vertaald: deze vergelijking is altijd onwaar, want een int is nooit null. Een waarschuwing die zegt dat iets altijd waar of altijd onwaar is, wijst bijna altijd op een fout.

Richting.

- Welk getal geeft Count bij een lege lijst? Met dat getal moet je vergelijken.
- Zet een breakpoint op regel 53, vraag een soort op die niet bestaat, en beweeg met je muis over artikelen. Visual Studio toont dan het aantal.
- Leerboek, hoofdstuk 6 (List), Wat je ermee kan: Count. Hoofdstuk 16 (Referentie tegenover waarde) en 17 (Omgaan met niets) leggen uit waarom een int nooit null is, en wat int? dan wel is.
- Er bestaat ook een methode die je rechtstreeks ja of nee geeft op de vraag "zit er iets in?": leerboek, hoofdstuk 11 (LINQ), bij Ik wil weten of iets bestaat. Beide manieren zijn goed.

A3. Punt 6: het endpoint voor wat op voorraad is, ontbreekt nog

Verzoek	Je API antwoordt nu	De opgave vraagt
GET api/artikel/voorradig	404 Not Found, zonder bericht (er is geen endpoint)	200 OK met enkel de artikelen met een voorraad groter dan 0

Richting.

- Dit lijkt sterk op punt 5: een lege lijst, een lus, en een voorwaarde om te bepalen welke artikelen erin komen. Alleen de voorwaarde is anders.
- Het grote verschil zit ná de lus. Bij punt 5 is een lege lijst een 404. Hier niet: als er niets op voorraad is, geef je de lege lijst gewoon terug, met 200. Dezelfde situatie, een ander antwoord. Dat is het leerdoel van de oefening: een leeg resultaat is soms een fout, en soms een geldig antwoord.
- Het adres: lees regel 3 van hoe ASP.NET een adres samenstelt nog eens. voorradig is een vast woord, geen {...}.

A4. Punt 7: de artikelfiche ontbreekt nog

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>api/artikel/fiche/1</code>	404 Not Found, zonder bericht (er is geen endpoint)	200 OK met de fiche van artikel 1
GET <code>api/artikel/fiche/99</code>	404 Not Found, zonder bericht	404 Not Found met <i>We vonden geen artikel met Id 99.</i>

Ook de klasse `ArtikelFiche` staat nog niet in de map `Models`.

Richting.

- Begin met de klasse. De vier gegevens en hun soort waarde staan in de tabel van punt 7. *Ja of nee* is in C# een `bool`. *Het artikel zelf* betekent dat het type van dat gegeven je eigen klasse `Artikel` is.
- Het endpoint moet drie vragen beantwoorden, en twee ervan heb je al eens opgelost:
 1. Bestaat het artikel? Dat is punt 4, met hetzelfde bericht als het niet bestaat.
 2. Hoeveel andere artikelen hebben dezelfde soort? Dat lijkt op punt 5.
 3. Is er een ander artikel van die soort dat goedkoper is?
- Twee valkuilen, uit de zinnen onder de tabel:
 - **Het artikel zelf telt niet mee.** Zoek je alle artikelen van dezelfde soort, dan zit het artikel zelf daar ook tussen. Telt je fiche bij een artikel dat als enige van zijn soort is 1 in plaats van 0, dan heb je het artikel zelf meegeteld.
 - **"Geen enkel ander artikel is goedkoper"** is iets anders dan "dit is het allergeodkoopste". Hebben twee artikelen dezelfde laagste prijs, dan is bij allebei geen ander artikel goedkoper, dus staat het bij allebei op ja. In je huidige lijst komt dat niet voor, maar denk er wel aan bij het testen.
- `IsVoorradig` volgt dezelfde regel als punt 6: een voorraad groter dan 0.
- Na de drie vragen maak je één nieuw `ArtikelFiche`-object, vul je de vier gegevens in, en geef je dat terug. Het returntype van de methode is dus een fiche, geen artikel.

A5. Punt 5: de soortroute werkt niet meer — nieuw in V2

Waar: `ArtikelController.cs`, het attribuut boven `GetArtikelSoort` (regel 39).

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>api/artikel/soort/Elektronica</code>	404 Not Found, zonder bericht	200 OK met de artikelen van die soort
GET <code>api/artikel/soort/processor</code>	404 Not Found, zonder bericht	404 Not Found met <i>We vonden geen artikelen van de soort processor.</i>

Waarom. In V1 was deze route juist: ze kwam achter de controllerroute en gaf `api/Artikel/soort/{soort}`. Nu begint ze met een `/`. Volgens regel 4 valt de controllerroute dan weg, en luistert je endpoint op `artikel/soort/{soort}`, zonder `api` ervoor.

Richting. Je V1-route voor dit punt was goed. Zet die naast de route die je nu hebt, en kijk wat je veranderd hebt. Pas je een route aan, controleer dan in `Scalar` meteen of het volledige adres nog klopt.

LET OP

Een route aanpassen tot het werkt, is gissen. Bereken eerst met de vier regels hierboven welk adres eruit komt, en pas dan je route aan. Zo voorkom je dat een correcte route stukgaat, zoals hier.

B. Werkt, maar wijkt af

B1. GetArtikelSoort belooft één artikel, maar geeft een lijst

Waar: ArtikelController.cs, de kop van GetArtikelSoort (regel 40).

Wat de client krijgt, klopt: een JSON-lijst. Maar het returntype tussen de punthaken zegt dat er **één** Artikel terugkomt.

- **Waarom C# het toelaat.** `Ok(...)` aanvaardt elke waarde, en ASP.NET zet het resultaat om naar je returntype zonder te kijken wat erin zit. Er komt dus geen fout en geen waarschuwing.
- **Waarom het toch telt.** Het returntype is een belofte aan wie je API gebruikt. Scalar leest dat type en toont daardoor één artikel als voorbeeld van het antwoord, terwijl je een lijst teruggeeft. Wie op je documentatie vertrouwt, wordt dus misleid. En wie je code nakijkt, ziet het meteen, ook als de JSON klopt.

Richting. Vergelijk met `GetArtikelen` op regel 20: dat geeft ook een lijst terug. Kijk wat daar tussen de punthaken staat.

C. Opkuis, zonder invloed op het antwoord

Waar	Wat	Richting
regel 29	Waarschuwing CS8600: <i>Converting null literal or possible null value to non-nullable type.</i> <code>FirstOrDefault</code> kan <code>null</code> geven als er niets gevonden wordt, maar het type van je variabele zegt dat ze nooit <code>null</code> kan zijn. Je code werkt, want je controleert daarna op <code>null</code> , maar de compiler weet dat niet.	Kijk hoe de les die variabele schrijft, in de <code>ProductController</code> van je theorieproject: daar staat geen type, maar een woord dat C# het type zelf laat bepalen. Of lees in je leerboek, hoofdstuk 17, wat een vraagteken achter een type betekent.
regel 8 en 43	Het veld heet <code>Artikelen</code> , de lijst in <code>GetArtikelSoort</code> heet <code>artikelen</code> . Ze verschillen enkel in één hoofdletter, en dat maakt ze makkelijk te verwarren, zeker op regel 45, waar ze allebei in de buurt staan. Velden van een klasse beginnen in C# met een kleine letter; de les schrijft <code>laptops</code> .	Kies voor de lijst in de methode een naam die zegt wat erin zit, bijvoorbeeld wat ze verzamelt.
<code>ProgrammingAdvancedOefeningen.http</code>	Het verwijst nog naar <code>/weatherforecast</code> , een overblijfsel van het sjabloon.	Weghalen, of vervangen door de verzoeken uit de opgave: dan kan je ze in Visual Studio met één klik opnieuw doen.
regel 17-18 en 62-63	Twee lege regels na elkaar.	Eén lege regel tussen twee methodes is genoeg.

Waar je de waarschuwingen ziet. In Visual Studio: menu *View* → *Error List*, en zet daar *Warnings* aan. Je ziet ze ook als een groene golflijn onder de code. Lees ze na elke build: de fout in punt 5 (A2) had de compiler al opgemerkt.

D. Wat beter is dan in V1

- Je controleert in punt 5 nu het **aantal** in de lijst. Dat is het juiste gegeven om te bekijken.
- Je hebt bij beide routes gezien dat het adres niet klopt, en je hebt ze geprobeerd te verbeteren. De richting is goed; de regels hierboven helpen je om het in één keer juist te zetten.

E. Zelf nakijken

Doe na het oplossen deze verzoeken, in deze volgorde. Een fout is pas opgelost als het antwoord letterlijk overeenkomt met de opgave: statuscode én bericht.

Verzoek	Wat je moet zien	Hoort bij
Scalar openen	elk endpoint begint met <i>api/Artikel</i>	A1, A5
GET <code>api/artikel</code>	200 met je zes artikelen, zoals nu	punt 3
GET <code>api/artikel/1</code>	200 met artikel 1	A1
GET <code>api/artikel/99</code>	404 met <i>We vonden geen artikel met Id 99.</i>	A1
GET <code>api/artikel/soort/Elektronica</code>	200 met enkel artikelen van die soort	A5
GET <code>api/artikel/soort/processor</code>	404 met <i>We vonden geen artikelen van de soort processor.</i>	A2
Error List na een build	geen waarschuwing meer op regel 53	A2
GET <code>api/artikel/voorrading</code>	200, en geen enkel artikel met voorraad 0 in het antwoord	A3
idem, met tijdelijk alle voorraden op 0	200 met een lege lijst, geen 404	A3
GET <code>api/artikel/fiche/{id}</code>	de drie controles onderaan punt 7 van de opgave	A4
GET <code>api/artikel/fiche/99</code>	404 met <i>We vonden geen artikel met Id 99.</i>	A4

Met het nakijksript, als je API in Visual Studio gesloten is:

```
.\Nakijk.ps1 -Oefening 02_01 -Project  
"... \ProgrammingAdvancedOefeningen \ProgrammingAdvancedOefeningen"
```

Het script start je API zelf en stuurt de verzoeken uit de opgave. Het verslag komt in de map **Nakijken** naast je project.

Vragen die je je misschien stelt

**Waarom werkt artikel/artikel/1 nu wel in mijn browser, maar zegt de nakijk dat het fout is? Omdat de opgave een ander adres vraagt: api/artikel/1*. Een client, of een examiner, roept het adres uit de opgave op. Dat je endpoint op een ander adres wel antwoordt, helpt dan niet.*

**Is de hoofdletter in api/Artikel een probleem? Nee. Adressen zijn niet hoofdlettergevoelig: api/artikel* komt op hetzelfde endpoint uit.*

**Kunnen {id} en soort/{soort} elkaar in de weg zitten? Nee. Ze hebben een verschillend aantal stukken: api/artikel/5 heeft er drie, api/artikel/soort/Audio vier. Daar gaat de waarschuwing in punt 5 van de opgave over: had je de soortroute zonder soort/* geschreven, dan zouden ze elkaar wel in de weg zitten.*

**En voorradig en {id}? Die hebben allebei drie stukken. ASP.NET geeft dan voorrang aan het vaste woord voorradig boven een {...}. Dat gaat dus goed, zolang je voorradig* als vast woord in de route zet.*

Waarom moet een onbekende soort een 404 geven, maar een lege voorraad niet? Omdat de vraag anders is. Wie een soort opvraagt die niet bestaat, vraagt naar iets dat er niet is: dat is een fout van de client, dus 404. Wie vraagt wat er op voorraad is, stelt een geldige vraag waarop het antwoord "niets" kan zijn: dat is 200 met een lege lijst. De opgave noemt dat zelf het leerdoel.

**Moet de soort hoofdletterongevoelig zijn? elektronica geeft bij mij niets. De opgave vraagt het niet: je vergelijkt nu exact, en dat is goed. Wil je het toch, dan staat in je leerboek, hoofdstuk 1 (Tekst), bij Tekst vergelijken*, hoe je twee teksten vergelijkt zonder op hoofdletters te letten. Doe dat pas als de rest werkt.*

Waarom staat mijn lijst `Artikelen` niet `static`, terwijl de eindversie van les 02 dat wel doet? Voor deze oefening maakt dat niet uit: je leest de lijst enkel. Pas wanneer je iets toevoegt of wijzigt, zoals in 02_02, merk je het verschil. Dat komt daar aan bod.

Buiten deze oefening

Niet nagekeken, maar opgevallen: in de `OnderdelenController` heet de routeparameter van het codeendpoint anders dan de parameter van de methode. Dat is hetzelfde mechanisme als fout 1 tot 3 in 01_03: de naam tussen de accolades koppelt alleen aan een parameter met precies dezelfde naam.